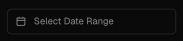
Badge
Banner
Book
Breadcrumbs
Browser
Button
Calendar
Checkbox
Choicebox
Clearable Input
Code
Code Block
Collapse
Combobox
Command Menu
Context Card
Context Menu
Copy Button
Description
Destructive Action Modal
Destructive Confirmation Input
Dots Menu
Drawer
Empty State
Entity
Error
Error Card
Feedback

Calendar

Displays a calendar from which users can select a date or range of dates.

Default



Hide code

```tsx
import type { DateValue } from '@vercel/geistcn/components';
import { Calendar } from '@vercel/geistcn/components';
import { useState, type JSX } from 'react';
import type { RangeValue } from '@react-types/shared';

export function Component(): JSX.Element {
  const [date, setDate] = useState<RangeValue<DateValue>>();

  const now = new Date();

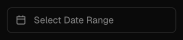
  const minDate = new Date(
    now.getFullYear(),
    now.getMonth() - 2,
    now.getDate(),
  );

  const maxDate = new Date(
    now.getFullYear(),
    now.getMonth() + 2,
    now.getDate(),
  );

  return (
    <div className="flex justify-center py-12">
      <Calendar
        allowClear
        onChange={setDate}
        value={date}
        minValue={minDate}
        maxValue={maxDate}
      />
    </div>
  );
}
```

Horizontal Layout

Use `horizontalLayout` to align content horizontally within the calendar popover.



Hide code

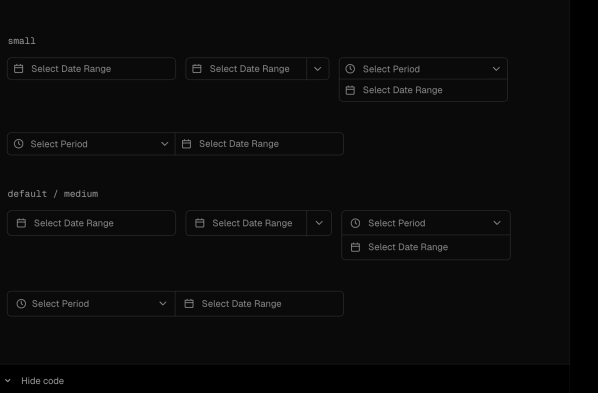
```tsx
import type { DateValue } from '@vercel/geistcn/components';
import { Calendar } from '@vercel/geistcn/components';
import { useState, type JSX } from 'react';
import type { RangeValue } from '@react-types/shared';

export function Component(): JSX.Element {
  const [date, setDate] = useState<RangeValue<DateValue>>();

  return (
    <div className="flex justify-center py-12">
      <Calendar
        allowClear
        onChange={setDate}
        value={date}
        horizontalLayout
        showTimeInput={false}
        popoverAlignment="center"
      />
    </div>
  );
}
```

Sizes

Choose between `medium` (default) and `small` for size.



Hide code

```tsx
import type { DateValue } from '@vercel/geistcn/components';
import { Calendar } from '@vercel/geistcn/components';
import { useState, type JSX } from 'react';
import type { RangeValue } from '@react-types/shared';
import { startOfDay, subDays, subWeeks, subMonths, endOfDay } from 'date-fns';

export function Component(): JSX.Element {
  const [date, setDate] = useState<RangeValue<DateValue>>();
  const [date2, setDate2] = useState<RangeValue<DateValue>>();

  const now = new Date();

  const minDate = new Date(
    now.getFullYear(),
    now.getMonth() - 2,
    now.getDate(),
  );

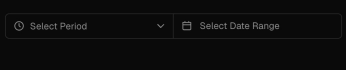
  const maxDate = new Date(
    now.getFullYear(),
    now.getMonth() + 2,
    now.getDate(),
  );

  const presets = {
    'last-3-days': {
      text: 'Last 3 Days',
      start: startOfDay(subDays(new Date(), 3)),
      end: endOfDay(new Date()),
    },
    'last-7-days': {
      text: 'Last 7 Days',
      start: startOfDay(subWeeks(new Date(), 1)),
      end: endOfDay(new Date()),
    },
    'last-14-days': {
      text: 'Last 14 Days',
      start: startOfDay(subWeeks(new Date(), 2)),
      end: endOfDay(new Date()),
    },
    'last-month': {
      text: 'Last Month',
      start: startOfDay(subMonths(new Date(), 1)),
      end: endOfDay(new Date()),
    },
  };

  return (
    <div className="py-12 space-y-12">
      <div>
        <p className="text-copy-14 text-gray-900 mb-4 font-mono">small</p>
        <div className="flex flex-wrap items-start gap-x-4 gap-y-12">
          <Calendar
            allowClear
            size="small"
            onChange={setDate2}
            value={date2}
            minValue={minDate}
            maxValue={maxDate}
          />
          <Calendar
            allowClear
            compact
            size="small"
            onChange={setDate2}
            value={date2}
            minValue={minDate}
            maxValue={maxDate}
            presets={presets}
          />
          <Calendar
            allowClear
            stacked
            size="small"
            onChange={setDate2}
            value={date2}
            minValue={minDate}
            maxValue={maxDate}
            presets={presets}
          />
          <Calendar
            size="small"
            onChange={setDate}
            presets={presets}
            value={date}
          />
        </div>
      </div>
      <div>
        <p className="text-copy-14 text-gray-900 mb-4 font-mono">
          default / medium
        </p>
        <div className="flex flex-wrap items-start gap-x-4 gap-y-12">
          <Calendar
            allowClear
            onChange={setDate2}
            value={date2}
            minValue={minDate}
            maxValue={maxDate}
          />
          <Calendar
            allowClear
            compact
            onChange={setDate2}
            value={date2}
            minValue={minDate}
            maxValue={maxDate}
            presets={presets}
          />
          <Calendar
            allowClear
            stacked
            onChange={setDate2}
            value={date2}
            minValue={minDate}
            maxValue={maxDate}
            presets={presets}
          />
          <Calendar onChange={setDate} presets={presets} value={date} />
        </div>
      </div>
    </div>
  );
}
```

Presets

Provide common date ranges.



Hide code

```tsx
import type { DateValue } from '@vercel/geistcn/components';
```

```tsx
import { Calendar } from '@vercel/geistcn/components';
import { useState, type JSX } from 'react';
import type { RangeValue } from '@react-types/shared';
import { startOfDay, subDays, subWeeks, subMonths, endOfDay } from 'date-fns';

const presets = {
  'last-3-days': {
    text: 'Last 3 Days',
    start: startOfDay(subDays(new Date(), 3)),
    end: endOfDay(new Date()),
  },
  'last-7-days': {
    text: 'Last 7 Days',
    start: startOfDay(subWeeks(new Date(), 1)),
    end: endOfDay(new Date()),
  },
  'last-14-days': {
    text: 'Last 14 Days',
    start: startOfDay(subWeeks(new Date(), 2)),
    end: endOfDay(new Date()),
  },
  'last-month': {
    text: 'Last Month',
    start: startOfDay(subMonths(new Date(), 1)),
    end: endOfDay(new Date()),
  },
};

export function Component(): JSX.Element {
  const [date, setDate] = useState<RangeValue<DateValue>>();

  return (
    <div className="flex justify-center py-12">
      <Calendar onChange={setDate} presets={presets} value={date} />
    </div>
  );
}
```

Compact



Hide code

```tsx
import type { DateValue } from '@vercel/geistcn/components';
import { Calendar } from '@vercel/geistcn/components';
import { useState, type JSX } from 'react';
import type { RangeValue } from '@react-types/shared';
import { startOfDay, subDays, subWeeks, subMonths, endOfDay } from 'date-fns';

const presets = {
  'last-3-days': {
    text: 'Last 3 Days',
    start: startOfDay(subDays(new Date(), 3)),
    end: endOfDay(new Date()),
  },
  'last-7-days': {
    text: 'Last 7 Days',
    start: startOfDay(subWeeks(new Date(), 1)),
    end: endOfDay(new Date()),
  },
  'last-14-days': {
    text: 'Last 14 Days',
    start: startOfDay(subWeeks(new Date(), 2)),
    end: endOfDay(new Date()),
  },
  'last-month': {
    text: 'Last Month',
    start: startOfDay(subMonths(new Date(), 1)),
    end: endOfDay(new Date()),
  },
};

export function Component(): JSX.Element {
  const [date, setDate] = useState<RangeValue<DateValue>>();

  return (
    <div className="flex justify-center py-12">
      <Calendar compact onChange={setDate} presets={presets} value={date} />
    </div>
  );
}
```

Stacked



Hide code

```tsx
import type { DateValue } from '@vercel/geistcn/components';
import { Calendar } from '@vercel/geistcn/components';
import { useState, type JSX } from 'react';
import type { RangeValue } from '@react-types/shared';
import { startOfDay, subDays, subWeeks, subMonths, endOfDay } from 'date-fns';

const presets = {
  'last-3-days': {
    text: 'Last 3 Days',
    start: startOfDay(subDays(new Date(), 3)),
    end: endOfDay(new Date()),
  },
  'last-7-days': {
    text: 'Last 7 Days',
    start: startOfDay(subWeeks(new Date(), 1)),
    end: endOfDay(new Date()),
  },
  'last-14-days': {
    text: 'Last 14 Days',
    start: startOfDay(subWeeks(new Date(), 2)),
    end: endOfDay(new Date()),
  },
  'last-month': {
    text: 'Last Month',
    start: startOfDay(subMonths(new Date(), 1)),
    end: endOfDay(new Date()),
  },
};

export function Component(): JSX.Element {
  const [date, setDate] = useState<RangeValue<DateValue>>();
```

```
  return (
    <div className="flex justify-center py-12">
      <Calendar onChange={setDate} presets={presets} stacked value={date} />
    </div>
  );
}
```

Presets with default value

Provide common date ranges with an additional default value.



Hide code

```
import type { DateValue } from '@vercel/geistcn/components';
import { Calendar } from '@vercel/geistcn/components';
import { useState, type JSX } from 'react';
import type { RangeValue } from '@react-types/shared';
import { startOfDay, subDays, subWeeks, subMonths, endOfDay } from 'date-fns';

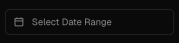
const presets = {
  'last-3-days': {
    text: 'Last 3 Days',
    start: startOfDay(subDays(new Date(), 3)),
    end: endOfDay(new Date()),
  },
  'last-7-days': {
    text: 'Last 7 Days',
    start: startOfDay(subWeeks(new Date(), 1)),
    end: endOfDay(new Date()),
  },
  'last-14-days': {
    text: 'Last 14 Days',
    start: startOfDay(subWeeks(new Date(), 2)),
    end: endOfDay(new Date()),
  },
  'last-month': {
    text: 'Last Month',
    start: startOfDay(subMonths(new Date(), 1)),
    end: endOfDay(new Date()),
  },
};

export function Component(): JSX.Element {
  const [date, setDate] = useState<RangeValue<DateValue>>();

  return (
    <div className="flex justify-center py-12">
      <Calendar
        onChange={setDate}
        presetIndex={2}
        presets={presets}
        stacked
        value={date}
      />
    </div>
  );
}
```

Min and max dates

Provide common date ranges with an additional default value.



Hide code

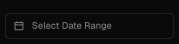
```
import type { DateValue } from '@vercel/geistcn/components';
import { Calendar } from '@vercel/geistcn/components';
import { useState, type JSX } from 'react';
import type { RangeValue } from '@react-types/shared';
import { addDays, subDays } from 'date-fns';

export function Component(): JSX.Element {
  const [date, setDate] = useState<RangeValue<DateValue>>();
  const minValue = subDays(new Date().setHours(5), 1); // 2 days ago
  const maxValue = addDays(new Date(), 1); // 2 days from now

  return (
    <div className="flex justify-center py-12">
      <Calendar
        maxValue={maxValue}
        minValue={minValue}
        onChange={setDate}
        value={date}
      />
    </div>
  );
}
```

Pinned timezone

Use `pinnedTimezone` to lock the calendar to a specific timezone. The timezone is shown as read-only text instead of a select.



Hide code

```
import type { DateValue } from '@vercel/geistcn/components';
import { Calendar } from '@vercel/geistcn/components';
import { useState, type JSX } from 'react';
import type { RangeValue } from '@react-types/shared';
```

```tsx
export function Component(): JSX.Element {
  const [date, setDate] = useState<RangeValue<DateValue>>();

  return (
    <div className="flex justify-center py-12">
      <Calendar
        onChange={setDate}
        pinnedTimezone="America/Los_Angeles"
        popoverAlignment="center"
        value={date}
      />
    </div>
  );
}
```

Best Practices

When to use

- Pick `<Calendar>` for analytics ranges and any picker where day-of-week and month context matter.
- For ISO dates pasted whole or relative shorthand like `7d`, use a free-form **Input**.
- Provide `<Calendar.Presets>` for the common ranges (`Last 7 Days`, `Month to Date`) so users land on the right window in one click.
- Pair a horizontal layout with live results next to the calendar; in narrow surfaces like a sidebar, fall back to the stacked layout.

Behavior

- Set `min` and `max` to the data window so users can't pick outside the retention range.
- Default to the user's locale and timezone; never silently render UTC for a US-Pacific viewer.
- Keep the trigger label as the chosen range (`Apr 1 - Apr 28, 2026`); don't fall back to `Pick a date` once a value is committed.
- Persist the selected range when the popover closes and re-opens so users can tweak the end date without re-picking the start.

Accessibility

- Trap focus inside the popover so Tab cycles day cells and presets instead of the page behind it.
- Support arrow-key day navigation, `Shift` + arrow for week jumps, and `Page Up` / `Page Down` for month jumps.
- Announce range changes through `aria-live="polite"` so a screen reader hears `From Apr 1 to Apr 28` after the second click.
- Each preset is a real button with a Title Case label (`Last 30 Days`); don't mark presets as menu items without keyboard handling.